David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

Advanced Word Embeddings and Evaluation

① Gradient descent, revisited
② Skip-gram with negative sample
③ Window-based representation
④ Count-based vs. direct prediction
⑤ Embedding meanings, word sense
⑥ How to evaluate word vectors

Lecture Objectives

1. Highlight some logistics
2. Gradient descent example
3. Skip-gram with negative sampling
4. Window-based model
5. Evaluation techniques and word sense

Logistics

1

Modules 2 is released

2

Proposal will be graded shortly

5

Office hour time; 2:00pm – 3:00pm, Tuesdays
Away: Feb 6 to Feb 12

3

Academic activity is due on Jan 23, 2026,
grade counts towards the bonus

4

Assignment 1 is due on Jan 26, 2026

Gradient Descent

Gradient Descent

- $f(x) = x^2$, $f'(x) = 2x$, $f'(0) = 0$.
- Initial guess $x = 4$; $f'(x) = 2*4 = 8$
- Update: $x_{new} = x_{old} - a\, f'(x_{old})$

$x_{new} = 4 - 0.1 * 8 - 3.2$

$f(3.2) = 2*3.2 = 6.4$

…Repeat: $x_{new} = 3.2 - 0.1*6.4 = 2.56$; f(x) = 2*2.56=5.12

…

Update equation (in matrix notation):

$$\theta^{new} = \theta^{old} - \alpha \nabla_\theta J(\theta)$$

α = *step size* or *learning rate*

Update equation (for single parameter):

$$\theta_j^{new} = \theta_j^{old} - \alpha \frac{\partial}{\partial_j^{old}} J(\theta)$$

Gradient Descent

- Define variables:
  - Predictions (vector of predicted values): $\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \hat{y}_3]$
  - Target (actual values): $\mathbf{y} = [y_1, y_2, y_3].$
  - The function is MSE loss; n = 3: $L(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{n}\sum^{n}(\hat{y}_i - y_i)^2$
- Compute derivative w.r.t. the prediction. $\frac{\partial L}{\partial \hat{\mathbf{y}}} = \frac{2}{n}(\hat{\mathbf{y}} - \mathbf{y})$
- Substitute values; let's say:
  - Compute difference:
  - Compute the gradient:

$$\hat{\mathbf{y}} = [2.0, 3.0, 4.0]$$

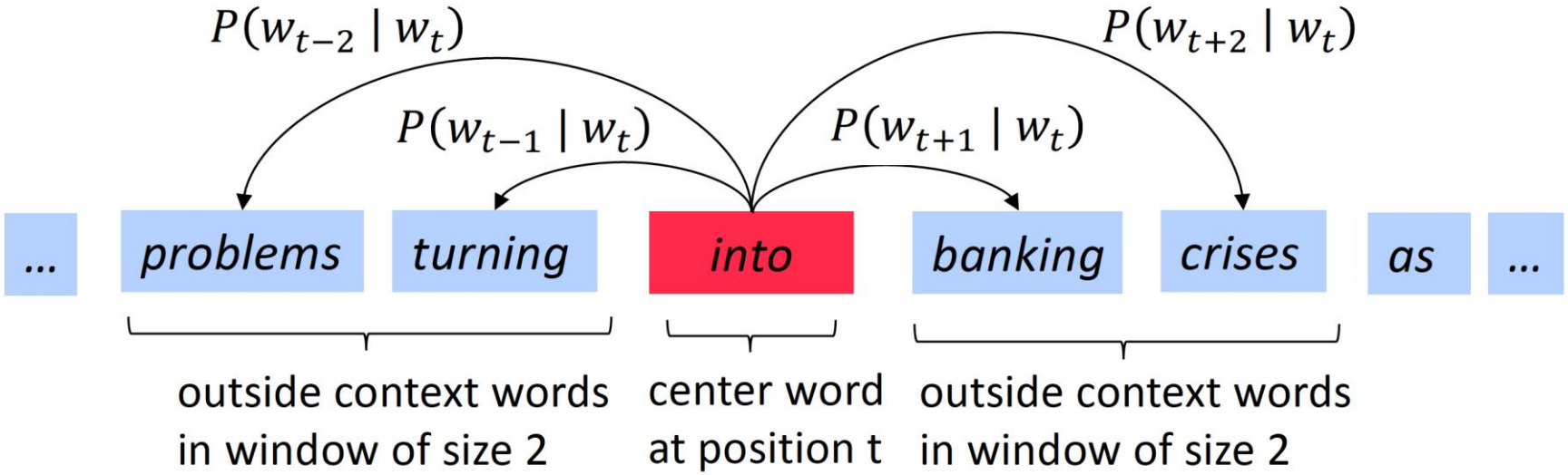
$$\mathbf{y} = [1.5, 3.5, 4.5]$$

$$\hat{\mathbf{y}} - \mathbf{y} = [2.0 - 1.5, 3.0 - 3.5, 4.0 - 4.5] = [0.5, -0.5, -0.5]$$

$$\frac{\partial L}{\partial \hat{\mathbf{y}}} = \frac{2}{3}[0.5, -0.5, -0.5] = [\frac{1}{3}, -\frac{1}{3}, -\frac{1}{3}]$$

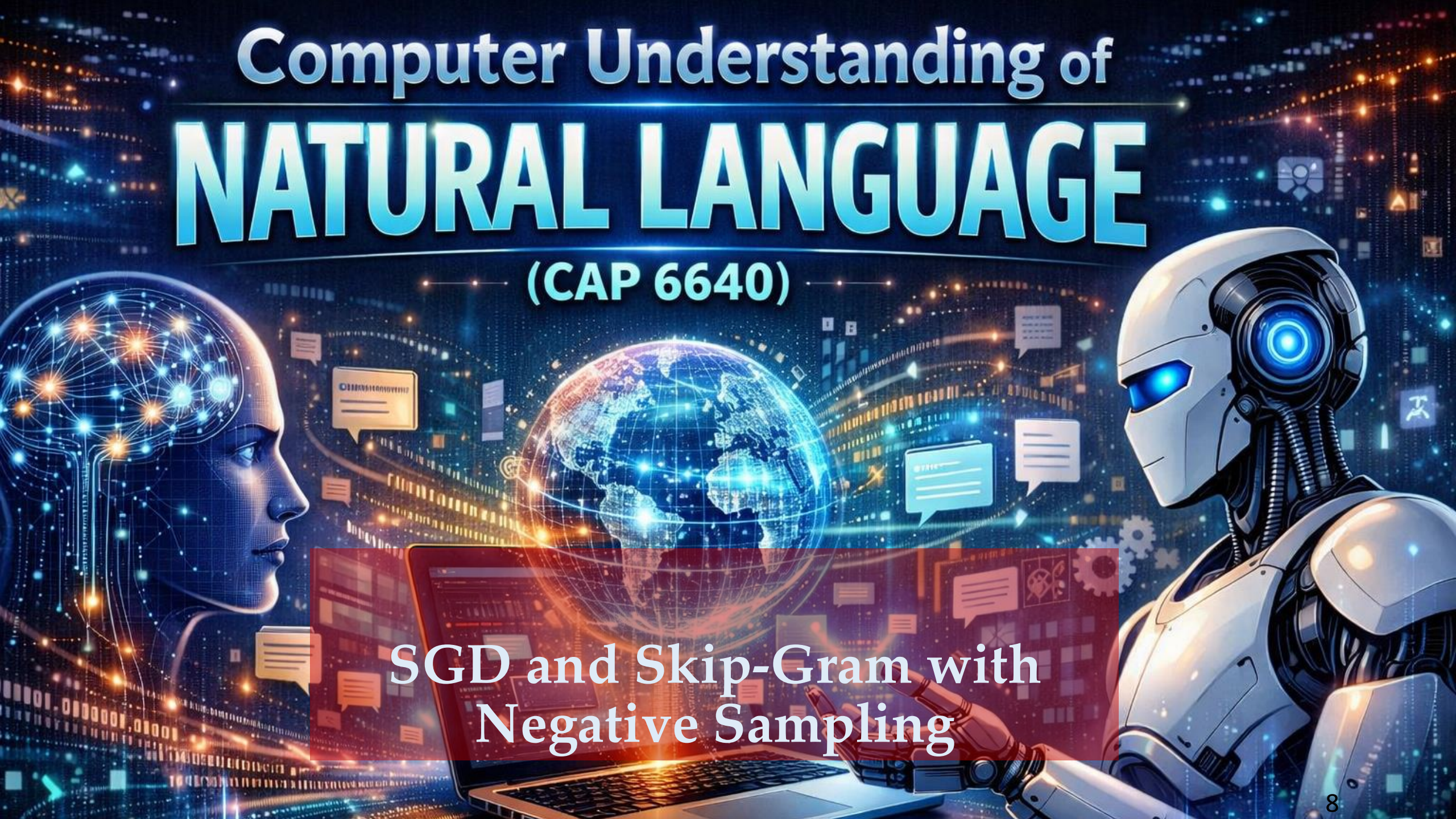
Summary of Word2Vec

- Iterate over each word for the whole corpus
- Predict surrounding words using word vectors



$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

- Update vectors so you can predict well



SGD and Skip-Gram with Negative Sampling

Stochastic Gradient Descent

- Problem: $J(\boldsymbol{\theta})$ is a function of all windows in the corpus (potentially billions)
  - So the update of $J(\boldsymbol{\theta})$ over $\boldsymbol{\theta}$ is expensive

- You would wait a very long time before making a single update.
  - Very bad idea for pretty much all problems

- Solution: SGD: repeatedly sample windows, and update after each one.

Skip-gram with Negative Sample

- The normalization is too expensive

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

- Hence, we implement the skip-gram

- **Idea**: train a binary logistic regression for a true pair (center with actuals) versus several noise pairs (center with random words)

Skip-Gram with Negative Sampling

- Overall objective function to maximize

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J_t(\theta)$$

$$J_t(\theta) = \log \sigma \left(u_o^T v_c\right) + \sum_{i=1}^{k} \mathbb{E}_{j \sim P(w)} \left[\log \sigma \left(-u_j^T v_c\right)\right]$$

- Meaning: we maximize two words co-occurring in first log.

Skip-Gram: Procedure

- We take k negative samples (using the word probabilities)

- Maximize the probability that real outside word appears, minimize prob. that random words appear around the center word.

- Why not capture co-occurrence directly:
  - Window: similar to Word2Vec, use window around each word ⇔ capture both syntactic and semantic information.
  - Word-document co-occurrence matrix gives general topics (all sports terms will have similar entities) leading to ”latent semantic analysis”.

Skip-gram with Negative Sample: Details

- **Objective:** Learn dense embeddings by predicting context words given a target.
- **How It Works:**
  1. **Input:** Target word and its context window.
  2. **Positive Samples:** Word pairs from the corpus (w_{target}, $w_{context}$).
  3. **Negative Samples:** Random pairs (w_{target}, w_{noise}) generated from a noise distribution.

- **Training:** Binary classification to distinguish real (+) from random (-) pairs.
- **Output:** Embeddings that capture semantic similarity.

- **Advantages:**
  - Efficient for large vocabularies.
  - Captures semantic relationships (e.g., analogies, similarities).
  - Widely used in NLP tasks (e.g., translation, sentiment analysis).

- **Apps:** Pre-trained embeddings for downstream NLP tasks, semantic search, etc.

Window-based Representation

Example: Window-Based

- Window length = 1 (common 5-10)

- Symmetric (irrelevant whether left or right context).

- Example corpus:
  - I like deep learning
  - I like NLP
  - I enjoy flying

Example: Window-Based, cont.

- Example corpus:
  - I like deep learning
  - I like NLP
  - I enjoy flying

counts	I	like	enjoy	deep	learning	NLP	.
I	0	2	1	0	0	0	0
like	2	0	0	1	0	1	0
enjoy	1	0	0	0	0	0	0
deep	0	1	0	0	1	0	0
learning	0	0	0	1	0	0	0
NLP	0	1	0	0	0	0	0
flying	0	0	0	0	0	1	1
.	0	0	0	0	1	1	0

Low-Dimensional Vector Representations

- **Key Idea:** Compress co-occurrence statistics into a fixed-size, dense representation that preserves the most informative structure.
- **Dense Word Vectors:**
  - Represent each word using a low-dimensional continuous vector.
  - Typical dimensionality: **25–1000**, consistent with models such as *word2vec*.

- **Dimensionality Reduction Approach:**
  - Apply **Singular Value Decomposition (SVD)** to the co-occurrence matrix X.
  - Factorize $X = U\Sigma V^\top$,where $U$and $V$are orthogonal matrices.
  - Retain the top $k$singular values to obtain a generalized, lower-rank approximation.

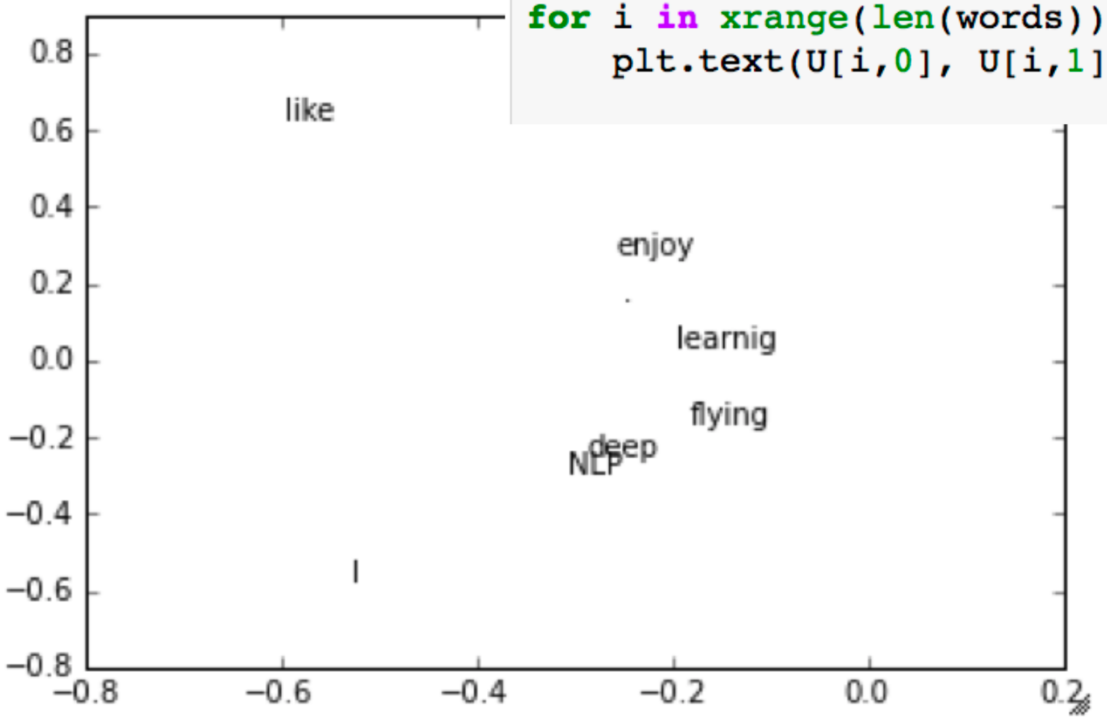
- **Limitations:**
  - SVD is computationally expensive for large, sparse co-occurrence matrices.
  - Scalability constraints motivate alternative learning-based methods.

Example in Python

```python
import numpy as np
# Linear algebra module
la = np.linalg
# Vocabulary
words = [
    "I", "like", "enjoy",
    "deep", "learning", "NLP",
    "flying", "."
]
# Co-occurrence matrix
X = np.array([
    [0, 2, 1, 0, 0, 0, 0, 0],
    [2, 0, 0, 1, 0, 1, 0, 0],
    [1, 0, 0, 0, 0, 0, 1, 0],
    [0, 1, 0, 0, 1, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 1],
    [0, 1, 0, 0, 0, 0, 0, 1],
    [0, 0, 1, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 1, 1, 1, 0]
])

# Singular Value Decomposition
U, S, Vh = la.svd(X, full_matrices=False)
```

```python
for i in xrange(len(words)):
    plt.text(U[i,0], U[i,1], words[i])
```



Printing first two columns of U corresponding to the biggest singular values

Hacks to X

- **Problem:** Function words (e.g., *the, he, has, a*) occur extremely frequently, causing syntactic patterns to dominate the statistics and dilute semantics.

- **Mitigation Strategies:**
  - **Count clipping:** Use min(X, t), t = 100 to limit the influence of very frequent words.
  - **Stop-word removal:** Ignore function words entirely.

- **Refinements:**
  - **Ramped context windows:** Assign higher weights to closer context words.
  - **Correlation-based measures:** Use Pearson correlation instead of raw counts, then set negative values to zero.
- **Open Question:** What other normalization or weighting strategies could further reduce syntactic bias while preserving semantic structure?

Count-Based vs. Direct-Prediction Models

- **Count-Based Models**
  - Fast to train
  - Efficient use of global co-occurrence statistics
  - Primarily capture word similarity
  - Disproportionate influence from very frequent words
- **Direct-Prediction Models**
  - Scale naturally with corpus size
  - Less efficient use of raw statistics
  - Achieve stronger performance on downstream tasks
  - Capture complex linguistic patterns beyond word similarity

Encoding Meaning

Encoding Meaning via Ratios of Co-Occurrence

- **Key Insight:** Meaning can be captured not by raw co-occurrence counts, but by **ratios of conditional probabilities** across contexts.
- **Example:** Comparing $P(x \mid ice)$ and $P(x \mid steam)$ reveals latent semantic at
  - **Physical s**
  - **Material**
- **Interpretat**
  - Large rati related terms).
  - Small rati -related terms).
  - Ratios nea e.g., *water*, random terms).

	x = solid	x = gas	x = water	x = random
$P(x\|\text{ice})$	large	small	large	small
$P(x\|\text{steam})$	small	large	large	small
$\frac{P(x\|\text{ice})}{P(x\|\text{steam})}$	large	small	~1	~1

- **Takeaway:** Relative co-occurrence patterns encode fine-grained semantic distinctions that absolute counts alone cannot capture.

Encoding Meaning as Linear Components

- **Question:** How can ratios of co-occurrence probabilities be represented as linear semantic components?
- **Answer: Log-Bilinear Models**
  - Represent words as vectors such that inner products encode log probabilities:

$$w_i \cdot w_j = \log P(i \mid j)$$

  - Use **vector differences** to capture **probability ratios**:

$$w_x \cdot (w_a - w_b) = \log \frac{P(x \mid a)}{P(x \mid b)}$$

- **Interpretation:**
  - Vector differences w_a-w_b encode **meaning contrasts** (e.g., *ice* vs. *steam*).
  - Dot products with w_x recover **log-ratios of co-occurrence**, isolating specific semantics.
  - Linear structure in the embedding space corresponds to interpretable semantic relations.
- **Takeaway:** Semantic meaning emerges as linear directions in the embedding space, enabling ratios of co-occurrence probabilities to be modeled geometrically.

Evaluating Vectors

Evaluating Word Vectors

- **Intrinsic Evaluation**
  - Targets a specific, isolated subtask (e.g., similarity, analogy)
  - Fast and inexpensive to compute
  - Useful for probing and understanding embedding behavior
  - Limited practical value unless correlated with downstream task performance

- **Extrinsic Evaluation**
  - Measures impact on a real end-to-end task (e.g., NER, parsing, sentiment)
  - Computationally expensive and time-consuming
  - Performance changes may be confounded by other system components
  - A strict comparison: if replacing *only* the embedding module improves accuracy, it is a clear win

Intrinsic Evaluation

- **Focus**: Measures the quality of embeddings on tasks that directly test word similarity or relatedness.

- **Methods**:
  - **Word Similarity Tasks**: (1) Use human-annotated datasets like WordSim-353 or SimLex-999. (2) Evaluate cosine sim. b/w vectors and compare to human judgments.
  - **Analogy Tasks**: (2) Solve analogies like "king - man + woman = queen." (2) Popular dataset: Google Analogy Dataset.
  - **Clustering**: Group similar words together and measure coherence.
  - **Qualitative Analysis**: Visualize embeddings using techniques like t-SNE or PCA.

- **Advantages**: (1) Quick/interpretable. (2) Independent of specific NLP apps.
- **Disadvantages**: May not reflect performance on downstream tasks.

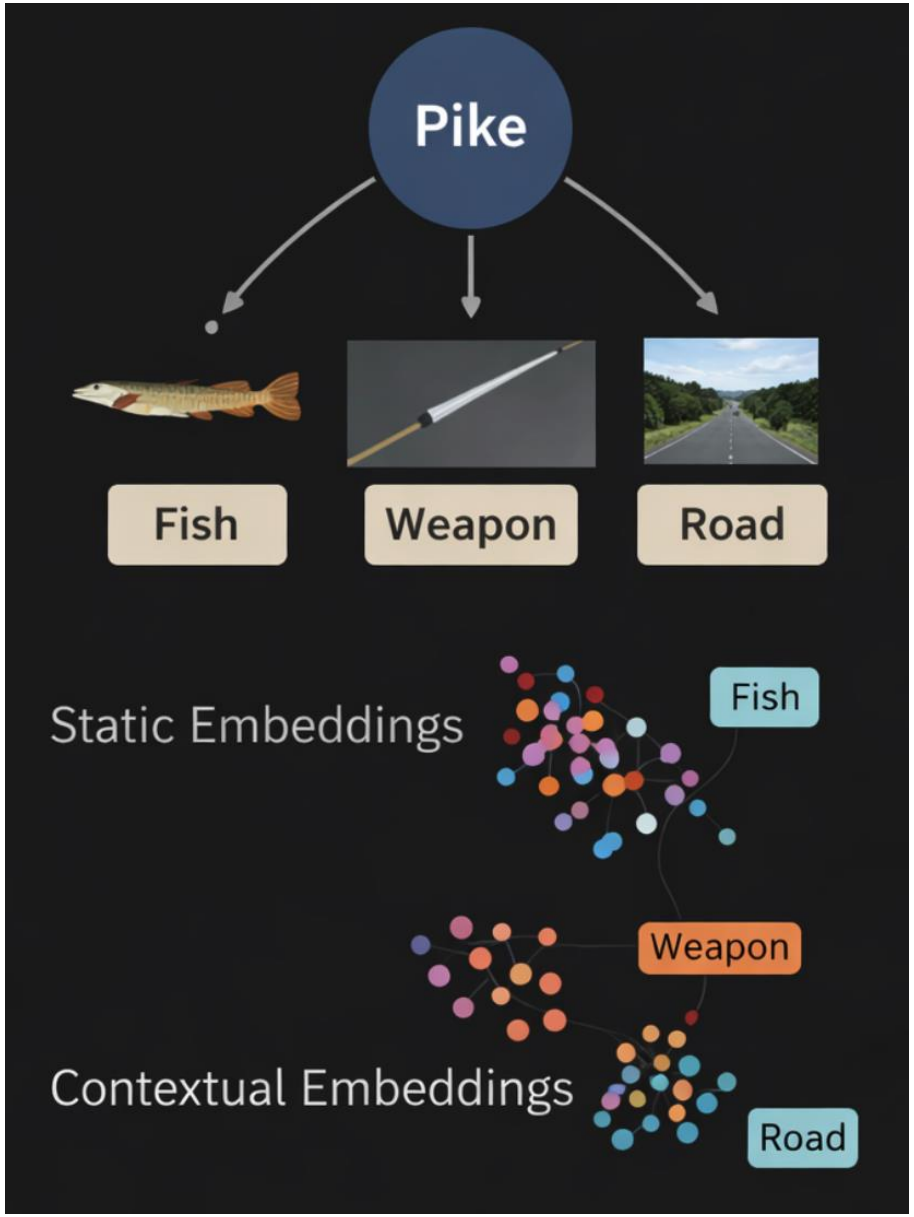
Extrinsic Evaluation

- **Focus:** Measures embedding performance in real-world NLP applications
- **Methods**
  - **Text Classification:** Use embeddings as input features (e.g., sentiment analysis, spam detection)
  - **Named Entity Recognition (NER):** Evaluate effectiveness in identifying entities (e.g., names, locations, dates)
  - **Machine Translation:** Assess contribution to translation quality
  - **Question Answering:** Test ability to support retrieval of correct answers
- **Advantages**
  - Directly measures practical utility
  - Reflects real-world task performance
- **Disadvantages**
  - Computationally expensive
  - Results are task- and system-specific

WordSense

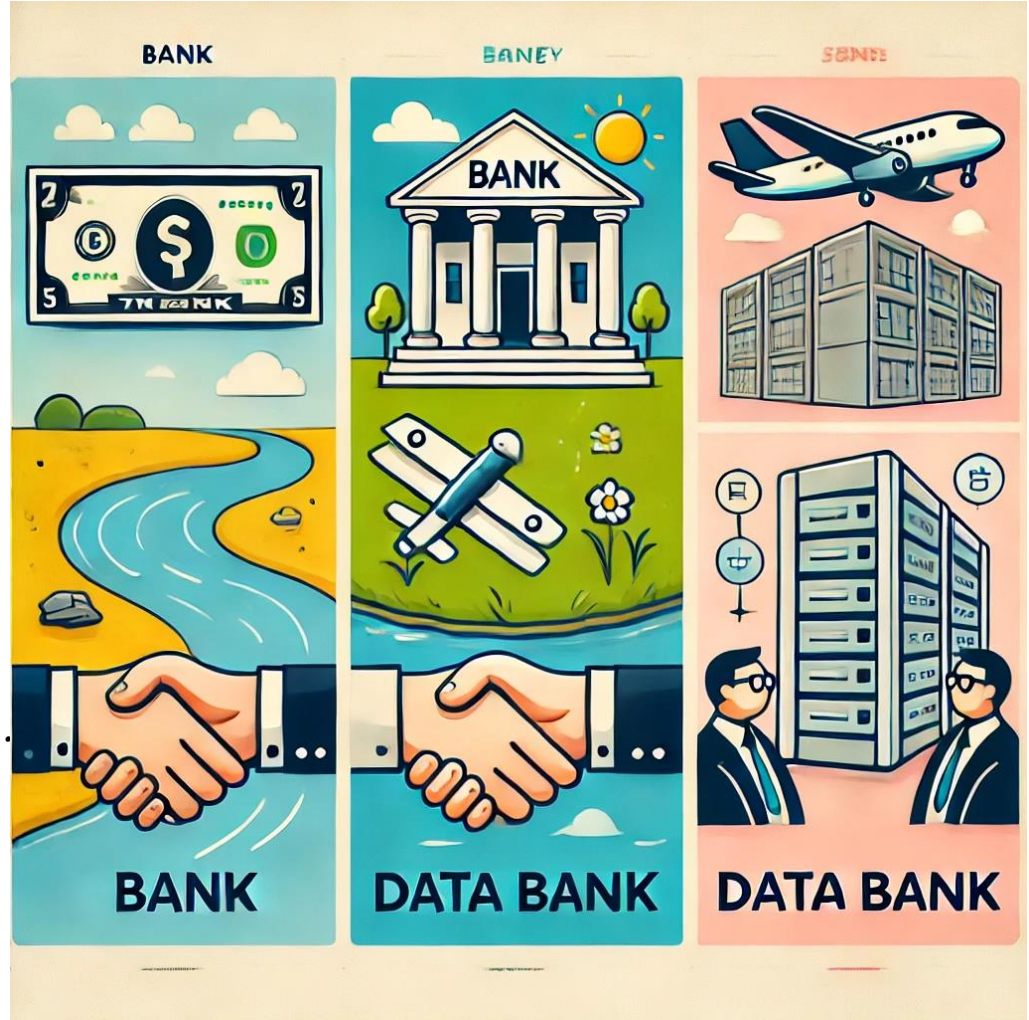
Word Sense

- **Words Have Multiple Meanings**
  - Ambiguity is common, especially for frequent or historical words.
  - Example: "Pike" (fish, weapon, road).
- **Challenge with Word Embeddings**
  - Static embeddings (e.g., Word2Vec) assign one vector per word.
  - Do they capture all meanings, or create a mess?
- **Solution: Contextual Embeddings**
  - Models like BERT assign different vectors based on context.
  - Helps resolve ambiguity dynamically.



Word Sense, example

- **Word: "Bank" – 5 Different Meanings**
  1. **Financial Institution**
     1. *"I deposited money in the bank."*
     2. Meaning: A place to store and manage money.
  2. **Riverbank**
     1. *"He sat on the bank of the river."*
     2. Meaning: The land beside a river.
  3. **Action of Banking (Tilting)**
     1. *"The airplane banked sharply to the left."*
     2. Meaning: To tilt or turn while in motion.
  4. **Storage (e.g., Data Bank)**
     1. *"The company maintains a bank of customer information.*
     2. Meaning: A collection or repository.
  5. **Verb: Trust or Rely**
     1. *"You can bank on her to deliver the project."*
     2. Meaning: To trust or depend on someone.



Word Sense, Another Example (Pike)

- A sharp point or staff
- A type of elongated fish
- A railroad line or system
- A type of road
- The future (coming down the pike)
- A type of body position (as in diving)
- To kill or pierce with a pike
- To make one's way (pike along)
- In Australian English, pike means to pull out from doing
- something: I reckon he could have climbed that cliff, but he piked!

Global Context Solutions

- **Global meaning in language representation:** Local co-occurrence statistics capture **syntactic proximity**, but many NLP tasks require understanding **global semantic structure** across the entire text.

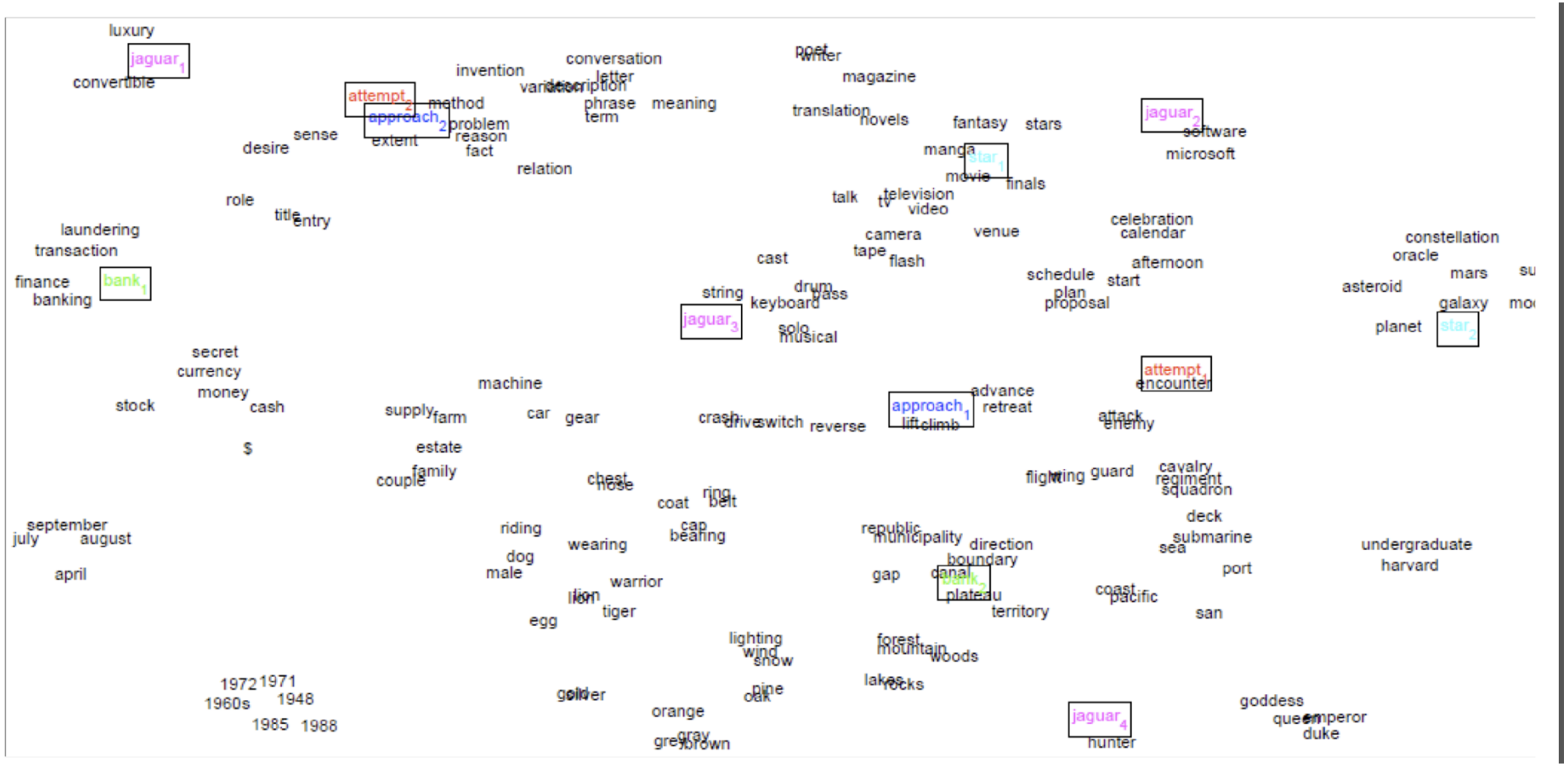
- Modern representation learning addresses this using the following mechanisms.
  - **Attention Mechanisms**
  - **Contextual Embeddings**
  - **Topic Models**

- **Attention Mechanisms**
  - Dynamically weight **relevant words or tokens** across the entire sequence.
  - Enable long-range dependencies without fixed context windows.
  - Form the core of Transformer architectures.
  - Allow models to focus on **semantically important context**, not just nearby words.
  - **Key idea:** Meaning emerges from selective interaction over the full sequence

Global Context Solutions, cont.

- **Contextual Embeddings**
  - Word representations depend on the **entire surrounding text**, not a fixed vector.
  - Same word can have **different embeddings** in different contexts.
  - Used in models such as BERT and GPT.
  - Capture syntax, semantics, and discourse-level information jointly.
  - **Key idea:** Words do not have a single meaning; meaning is context-dependent.

- **Topic Models**
  - Represent documents as mixtures of latent topics.
  - Capture **global thematic structure** across the text.
  - Useful for document-level understanding, clustering, and retrieval.
  - Provide interpretable global signals complementary to neural embeddings.
  - **Key idea:** Global meaning can be approximated by latent semantic themes.

Improving Word with Global Context

- Idea: cluster word windows around words, retain with each word assigned multiple different clusters.